KNN – K-Nearest Neighbors

Complete Notes: definition, working, distance, choosing K, use cases, advantages, disadvantages, scikit-learn model, practical example, evaluation, and interview questions.

1. What is K-Nearest Neighbors (KNN)?

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression. It predicts the output for a new data point by looking at the **K closest data points** in the training dataset.

Simple example: To predict whether a fruit is an apple or an orange, KNN compares its features (such as weight and size) with known fruits. If most of its nearest neighbors are apples, it predicts apple.

Classification: predicts a category (e.g., spam/not spam). **Regression:** predicts a numeric value (e.g., house price, often using the average of neighbors' values).

2. How does KNN work?

1. Store the labeled training data.

2. Choose a value for K (the number of neighbors).

3. Calculate the distance from the new point to training points.

4. Select the K points with the smallest distances.

5. For classification, use the majority class; for regression, commonly use the mean target value.

Important: KNN is often called a lazy learner because it does little model fitting during training; much of the work happens when making predictions.

3. Distance metrics

Metric	Idea / formula	When it can be useful
Euclidean	Straight-line distance: sqrt(sum((x■ – y■)²))	Common default for numeric features.
Manhattan	Sum of absolute differences: sum(|x■ – y■|)	Useful when distance along each feature dimension is preferred.
Minkowski	General distance family; p=1 gives Manhattan, p=2 gives Euclidean.	Flexible distance choice.

Distance depends on feature scales. For example, Salary may be measured in thousands while Age is measured in years. Without scaling, a large-range feature can dominate the distance calculation.

4. What does K mean? How do we choose K?

K is the number of nearest neighbors considered for a prediction.

K choice	Likely effect
Very small K (e.g., 1)	Sensitive to noise and individual examples; may overfit.
Very large K	Can smooth away useful local patterns; may underfit.
Practical approach	Try several values using cross-validation and choose the value with the best validation performance.

For binary classification, an odd K can reduce the chance of a tie, but it does not guarantee that ties are impossible in every multi-class problem.

5. When do we use KNN?

Area	Example
Healthcare	Classify cases based on similar patient records (with appropriate privacy and validation).
Banking	Identify transactions similar to known fraud patterns as one signal in a larger system.
Retail	Find customers with similar purchase behavior.
Image / pattern recognition	Classify observations using nearby feature vectors.
Regression	Estimate a numeric value from nearby examples, such as a price.

6. Advantages

• Simple to understand and implement.

• Can be used for both classification and regression.

• Makes few assumptions about the shape of the decision boundary.

• No explicit model-fitting step is required for basic KNN.

• Can work well on smaller datasets with meaningful, well-scaled features.

7. Disadvantages

• Prediction can be slow on large training datasets because distances must be compared.

• Uses memory to retain the training data.

• Sensitive to feature scaling and irrelevant features.

• Can perform poorly in high-dimensional data (the curse of dimensionality).

• Results depend on K and the distance metric.

• Imbalanced classes can bias majority voting toward the more common class.

8. KNN in Python (scikit-learn)

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)
```

Common arguments:

Argument	Meaning	Example
n_neighbors	Number of neighbors (K).	n_neighbors=5
weights	Uniform vote or closer points weighted more.	weights='uniform' / 'distance'
metric	Distance metric.	metric='minkowski'
p	For Minkowski: 1 = Manhattan, 2 = Euclidean.	p=2
n_jobs	Parallel jobs where supported.	n_jobs=-1

9. Complete practical example: Iris classification

This example uses the built-in Iris dataset, so no external CSV file is required. It predicts one of three flower species using four numeric measurements.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale features and train KNN in one pipeline
model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5, metric="minkowski", p=2)
)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Predict a new flower (measurements in cm)
new_flower = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(new_flower)[0]
print("Predicted flower:", iris.target_names[prediction])
```

Why use StandardScaler in a pipeline? KNN relies on distances, so features should be on comparable scales. The pipeline fits the scaler only on the training data and applies the same transformation to test and new data, avoiding data leakage.

10. Important methods and attributes

Item	Purpose
model.fit(X_train, y_train)	Store training examples / prepare estimator.
model.predict(X_test)	Predict class labels or numeric values.
model.predict_proba(X_test)	For classifiers, estimate class probabilities when supported.
model.score(X_test, y_test)	Default estimator score (accuracy for KNeighborsClassifier).
model.kneighbors(X)	Return neighbor distances and indices for compatible fitted estimators.

11. Evaluation metrics

Classification: accuracy, precision, recall, F1-score, and confusion matrix. For imbalanced classes, inspect precision/recall and per-class results rather than relying only on accuracy.

Regression: MAE (Mean Absolute Error), MSE (Mean Squared Error), RMSE (Root Mean Squared Error), and R^2 score.

Model selection: Use cross-validation to compare K values and, when relevant, distance metrics and weighting methods. Keep the final test set separate until final evaluation.

12. KNN vs Logistic Regression

Point	KNN	Logistic Regression
Approach	Instance-based; uses nearby observations.	Learns coefficients and a decision function.

Point	KNN	Logistic Regression
Training	Little fitting; retains data.	Optimization-based fitting.
Prediction	Can be costly on large datasets.	Usually fast after fitting.
Scaling	Usually important.	Often useful, especially with regularization.
Boundary	Can model nonlinear boundaries.	Linear boundary in the feature space by default.

13. Common mistakes to avoid

• Forgetting to scale numeric features.

• Scaling the full dataset before splitting; fit the scaler on training data only (a pipeline helps).

• Choosing K based only on training accuracy.

• Using irrelevant features that distort distances.

• Assuming KNN is always the best choice for large or high-dimensional datasets.

14. Key interview questions and answers

Q: What is KNN?
A: A supervised, instance-based algorithm that predicts using the K nearest training examples.

Q: Can KNN be used for regression?
A: Yes. A common approach is to average the target values of the nearest neighbors.

Q: Why is feature scaling important?
A: Distance calculations can be dominated by features with larger numeric ranges.

Q: What happens when K=1?
A: The prediction depends on the single nearest point and can be sensitive to noise.

Q: What happens when K is too large?
A: The prediction may become overly smooth and underfit local patterns.

Q: Why is KNN called a lazy learner?
A: It stores training data and postpones much of the computation until prediction.

Q: Which distance metrics are common?
A: Euclidean, Manhattan, and Minkowski.

Q: How can we select K?
A: Compare candidate values using cross-validation on training data.

Q: What is the curse of dimensionality?
A: In high dimensions, distances can become less informative and more data may be needed.

Q: Why use a Pipeline?
A: It ensures preprocessing is fitted on training folds/data and applied consistently, reducing leakage.

15. Quick revision

KNN = choose K → calculate distances → find nearest neighbors → majority vote (classification) or average (regression). Scale features, tune K with cross-validation, and evaluate on unseen test data.